

SKILLWEEK6

TASK1: Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
jhansi_priya@Jhansi-Priya: ~/  + - 
jhansi_priya@Jhansi-Priya:~$ mkdir Skill6
jhansi_priya@Jhansi-Priya:~$ cd Skill6
jhansi_priya@Jhansi-Priya:~/Skill6$ nano task1.c
jhansi_priya@Jhansi-Priya:~/Skill6$ gcc task1.c -o task1
jhansi_priya@Jhansi-Priya:~/Skill6$ ./task1
Enter commands (type 'history' to display, 'exit' to quit)
> ls
> pwd
> date
> whoami
> uname
> history

--- Command History ---
1. ls
2. pwd
3. date
4. whoami
5. uname
> command1
> command2
> command3
> command4
> command5
> command6
> history

--- Command History ---
1. command2
2. command3
3. command4
4. command5
5. command6
> exit
jhansi_priya@Jhansi-Priya:~/Skill6$ |
```

```
 jhansi_priya@Jhansi-Priya: ~/  + - 
GNU nano 7.2
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_CMD 100

char history[MAX_HISTORY][MAX_CMD];
int count = 0;

void add_command(char *cmd)
{
    if (count < MAX_HISTORY) {
        strcpy(history[count], cmd);
        count++;
    } else {
        // Remove oldest command
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], cmd);
    }
}

void display_history()
{
    printf("\n--- Command History ---\n");

    for (int i = 0; i < count; i++)
        printf("%d. %s\n", i + 1, history[i]);
}

int main()

^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^_ Replace    ^U Paste
^T ^J

29°C
```

```
GNU nano 7.2 task1.c *
    for (int i = 0; i < count; i++)
        printf("%d. %s\n", i + 1, history[i]);
}
int main()
{
    char command[MAX_CMD];

    printf("Enter commands (type 'history' to display, 'exit' to quit)\n");

    while (1) {
        printf("> ");

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            display_history();
            continue;
        }

        if (strlen(command) > 0)
            add_command(command);
    }

    return 0;
}
```

TASK2:Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <string.h>

#define MAX_COMMANDS 10
#define MAX_LENGTH 100

int main()
{
    char input[500];
    char *commands[MAX_COMMANDS];
    int count = 0;

    printf("Enter pipeline: ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, "|");

    while (token != NULL && count < MAX_COMMANDS) {
        commands[count++] = token;
        token = strtok(NULL, "|");
    }

    printf("\nPipeline contains %d commands:\n", count);

    for (int i = 0; i < count; i++) {
        printf("Command %d: %s\n", i + 1, commands[i]);
    }

    if (count == 1)
        printf("\nNo pipe detected.\n");
    else
        printf("\nPipeline layout is valid.\n");

    return 0;
}
```

```

jhansi_priya@Jhansi-Priya:~/Skill6$ nano task2.c
jhansi_priya@Jhansi-Priya:~/Skill6$ gcc task2.c -o task2
jhansi_priya@Jhansi-Priya:~/Skill6$ ./task2
Enter pipeline: ls | grep txt | wc -l

Pipeline contains 3 commands:
Command 1: ls
Command 2:  grep txt
Command 3:  wc -l

Pipeline layout is valid.
jhansi_priya@Jhansi-Priya:~/Skill6$ ls | grep txt | wc -l
0
jhansi_priya@Jhansi-Priya:~/Skill6$ |

```

TASK3: Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```

GNU nano 7.2 task3.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int fd[2];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid1 = fork();

    if (pid1 == -1) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {
        // First command: ls

        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("ls", "ls", NULL);
    }
}

```

```
GNU nano 7.2 task3.c *
    perror("execlp");
    exit(1);
}

pid_t pid2 = fork();
if (pid2 == -1) {
    perror("fork");
    return 1;
}

if (pid2 == 0) {
    // Second command: wc -l
    close(fd[1]);
    dup2(fd[0], STDIN_FILENO);
    close(fd[0]);
    execlp("wc", "wc", "-l", NULL);
    perror("execlp");
    exit(1);
}

// Parent does not need either end
close(fd[0]);
close(fd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

return 0;
}
```

```
0
jhansi_priya@Jhansi-Priya:~/Skill6$ nano task3.c
jhansi_priya@Jhansi-Priya:~/Skill6$ gcc task3.c -o task3
jhansi_priya@Jhansi-Priya:~/Skill6$ ./task3
6
jhansi_priya@Jhansi-Priya:~/Skill6$ |
```

TASK4: Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```
0
jhansi_priya@Jhansi-Priya:~/Skill6$ nano task4.c
jhansi_priya@Jhansi-Priya:~/Skill6$ gcc task4.c -o task4
jhansi_priya@Jhansi-Priya:~/Skill6$ ./task4
0
jhansi_priya@Jhansi-Priya:~/Skill6$ ls | grep txt | wc -l
0
jhansi_priya@Jhansi-Priya:~/Skill6$ |
```

```
jhansi_priya@Jhansi-Priya: ~/  + v
GNU nano 7.2 task4.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char *cmd1[] = {"ls", NULL};
    char *cmd2[] = {"grep", "txt", NULL};
    char *cmd3[] = {"wc", "-l", NULL};

    int pipe1[2];
    int pipe2[2];

    pipe(pipe1);
    pipe(pipe2);

    pid_t p1, p2, p3;

    // Process 1: ls
    p1 = fork();

    if (p1 == 0) {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execvp(cmd1[0], cmd1);

        perror("execvp");
        exit(1);
    }

    // Process 2: grep txt
    p2 = fork();

    if (p2 == 0) {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);

```

```
jhansi_priya@Jhansi-Priya: ~/  + v
GNU nano 7.2 task4.c *
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execvp(cmd2[0], cmd2);

        perror("execvp");
        exit(1);
    }

    // Process 3: wc -l
    p3 = fork();

    if (p3 == 0) {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execvp(cmd3[0], cmd3);

        perror("execvp");
        exit(1);
    }

    // Parent closes all pipe descriptors
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    // Wait for all processes
    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
    waitpid(p3, NULL, 0);

    return 0;
}
```

